Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Лабораторная работа №2
По вычислительной математике
Вариант №9

Выполнил:
Пчелкин Илья Игоревич
Группа № P3206
Проверил:
Зинченко Антон Андреевич

Санкт-Петербург 2026

Оглавление

Цель работы 3
Вычислительная реализация задачи 3
Решение нелинейного уравнения 3
Решение системы нелинейных уравнений 8
Программная реализация задачи 13
Листинг программы 13
Пример работы программы 21
Вывод 22

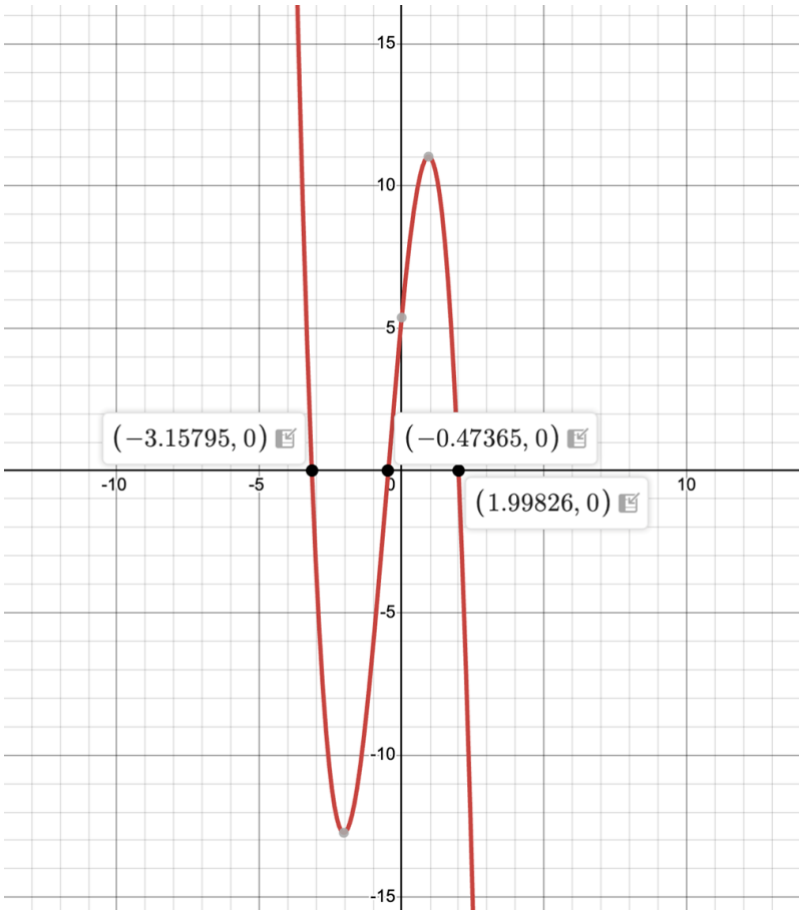
Цель работы

Изучить численные методы решения нелинейных уравнений и систем нелинейных уравнений

Вычислительная реализация задачи

Решение нелинейного уравнения

$$f(x) = -1.8x^3 - 2.94x^2 + 10.37 + 5.38$$



Интервалы изоляции корней

$x_l \in [-4; -3]$ – кр. левый корень

$x_m \in [-1; 0]$ – центр. корень

$x_p \in [1; 3]$ – кр. правый корень

$$-1,8x^3 - 2,94x^2 + 10,37x + 5,38 = 0$$

1) Найдем x_l методом простой итер.

$$x = \lambda(-1,8x^3 - 2,94x^2 + 10,37x + 5,38)$$

$$f'(x) = -1,8 \cdot 3x^2 - 2,94 \cdot 2x + 10,37$$

$$f'(a) = f'(-4) = -1,8 \cdot 3 \cdot (-4)^2 - 2,94 \cdot 2 \cdot (-4) + 10,37 =$$

$$= -52,51$$

$$f'(b) = f'(-3) = -1,8 \cdot 3 \cdot (-3)^2 - 2,94 \cdot 2 \cdot (-3) + 10,37 =$$

$$= -20,59$$

$$\Rightarrow \lambda = -\frac{1}{-20,59} = \frac{1}{20,59}$$

$$x = -\frac{1,8}{20,59}x^3 - \frac{2,94}{20,59}x^2 + \frac{10,37}{20,59}x + \frac{5,38}{20,59} + x$$

$x_0 = -4$

$x_1 = \varphi(x_0) = -\frac{1,8}{20,59}(-4)^3 - \frac{2,94}{20,59}(-4)^2 + \frac{10,37}{20,59}(-4) + \frac{5,38}{20,59} =$

$\approx -2,443$

$x_2 = \varphi(x_1) \approx -2,990$

$x_3 = \varphi(x_2) = -3,174$

$x_4 = -3,154$

$x_5 = -3,159$

№ итер.	x_k	x_{k+1}	$f(x_{k+1})$	$\lvert x_{k+1} - x_k \rvert$
1	-2,443	-2,990	-3,794	0,547
2	-2,990	-3,174	0,403	0,184
3	-3,174	-3,154	-0,098	0,020
4	-3,154	-3,159	0,026	0,005

2) Найдем x_m методом секущих

$x_0 = a = -1$

$x_1 = b = 0$

$x_2 = x_1 - f(x_1)\frac{x_1 - x_0}{f(x_1) - f(x_0)} =$

$$= 0 - 5{,}38 \frac{0-(-1)}{5{,}38-(-6{,}13)} \approx -0{,}467$$

$$x_3 = -0{,}467 - 0{,}079 \frac{-0{,}467-0}{0{,}079-5{,}38} \approx -0{,}474$$

N итер.	x_{k-1}	x_k	x_{k+1}	$f(x_{k+1})$	$\lvert x_{k+1}-x_k \rvert$
1	-1	0	$-0{,}467$	$0{,}079$	$0{,}468$
2	0	$-0{,}467$	$-0{,}474$	$-0{,}004$	$0{,}006$

3) Найдём x_r методом половин. дел.

$a = 1$

$b = 3$

$$x_0 = \frac{1+3}{2} = 2$$

$f(1)f(2) < 0 \Rightarrow a = 1$

$b = x_0 = 2$

$x_1 = \frac{1+2}{2} = 1,5$

$f(1)f(1,5) > 0 \Rightarrow a = x_1 = 1,5$

$b = 2$

$x_2 = \frac{1,5+2}{2} = 1,75$

$f(1,5)f(1,75) > 0 \Rightarrow a = x_2 = 1,75$

$b = 2$

$x_3 = \frac{1,75+2}{2} = 1,875$

$f(1,75)f(1,875) > 0 \Rightarrow a = 1,875$

$b = 2$

$x_4 = \frac{3,875}{2} \simeq 1,938$

$f(1,875)f(1,938) > 0 \Rightarrow a = 1,938$

$x_5 \simeq 1,969$

$f(1,938)(1,969) > 0 \Rightarrow a = 1,969$

$x_6 = 1,984$

$f(1,969)f(1,984) > 0 \Rightarrow a = 1,984$

$x_7 = 1,992$

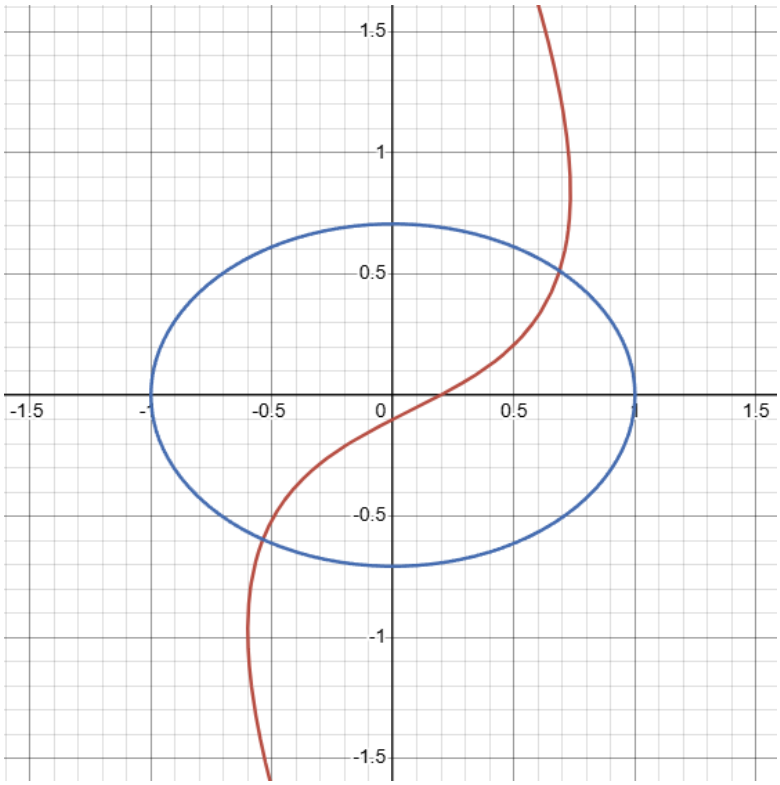
$f(1,984)f(1,992) > 0 \Rightarrow a = 1,992$

$x_8 = 1,996$

N шага	a	b	X	f(a)	f(b)	f(x)	|a − b|
1	1	3	2	11,010	−38,570	−0,040	2
2	1	2	1,5	11,010	−0,040	8,245	1
3	1,5	2	1,75	8,245	−0,040	4,877	0,5
4	1,75	2	1,875	4,877	−0,040	2,623	0,250
5	1,875	2	1,938	2,623	−0,040	1,344	0,125
6	1,938	2	1,969	1,344	−0,040	0,665	0,062
7	1,969	2	1,984	0,665	−0,040	0,316	0,031
8	1,984	2	1,992	0,316	−0,040	0,139	0,016
9	1,992	2	1,996	0,139	−0,040	0,050	0,008

Решение системы нелинейных уравнений

$$\begin{cases} \sin(x+y) = 1{,}5x - 0.1 \\ x^2 + 2y^2 = 1 \end{cases}$$



интервалы изоляции корней

$x_1 \in [0, 1]$ $y_1 \in [0, 1]$

$x_2 \in [0; -1]$ $y_2 \in [0; -1]$

решим систему методом Ньютона

$$\begin{cases} \sin(x+y) = 1,5x - 0,1 \\ x^2 + 2y^2 = 1 \end{cases}$$

$$\begin{cases} \sin(x+y) - 1,5x + 0,1 = 0 \\ x^2 + 2y^2 - 1 = 0 \end{cases}$$

пусть $x_0 = 0,7$ $y_0 = 0,5$, тогда

$$\begin{cases} x_1 = 0,7 + \Delta x \\ y_1 = 0,5 + \Delta y \end{cases} \Rightarrow \cancel{\sin 0,7+\Delta x}$$

$$\Rightarrow \begin{cases} \sin(0,7+\Delta x+0,5+\Delta y) - 1,5(0,7+\Delta x) + 0,1 = 0 \\ (0,7+\Delta x)^2 + 2(0,5+\Delta y)^2 - 1 = 0 \end{cases}$$

$\Leftrightarrow$

$$\begin{cases} \sin(\Delta x + \Delta y + 1,2) - 1,5\Delta x - 0,95 = 0 \\ \Delta x^2 + 1,4\Delta x + 2\Delta y^2 + 2\Delta y - 0,01 = 0 \end{cases}$$

$$\begin{cases} \sin(1,2) - 0,95 + (\cos(1,2) - 1,5)\Delta x + \cos(1,2)\Delta y + R = 0 \\ -0,01 + 1,4\Delta x + 2\Delta y + R = 0 \end{cases}$$

$$\begin{cases} -1,138\Delta x + 0,362\Delta y = 0,018 \\ 1,4\Delta x + 2\Delta y = 0,01 \end{cases}$$

Построим матрицу Якоби

$$\frac{\partial f}{\partial x} = \cos(x+y) - 1,5$$

$$\frac{\partial f}{\partial y} = \cos(x+y)$$

$$\frac{\partial g}{\partial x} = 2x$$

$$\frac{\partial g}{\partial y} = 4y$$

$$\begin{vmatrix} \cos(x+y)-1,5 & \cos(x+y) \\ 2x & 4y \end{vmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} =$$

$$= \begin{matrix} 1,5x - \sin(x+y) - 0,1 \\ 1 - x^2 - 2y^2 \end{matrix}$$

Найдем первую пару корней

Выберем $x_0 = 0,7$ $y_0 = 0,5$

$$\begin{cases} (\cos(x_0+y_0)-1,5)\Delta x + \cos(x_0+y_0)\Delta y = 1,5x_0 - \sin(x_0+y_0) - 0,1 \\ 2x_0\Delta x + 4y_0\Delta y = 1 - x_0^2 - 2y_0^2 \end{cases}$$

$\Delta x \approx -0,01$ $\Delta y \approx 0,01$

$x_1 = x_0 + \Delta x = 0,7 + (-0,011) = 0,69$

$y_1 = y_0 + \Delta y = 0,5 + 0,013 = 0,51$

проверим критерий окончания итер. процесса

$|0,689 - 0,7| \le 0,01$ $|0,513 - 0,5| \le 0,1$

$0,011 \le 0,01$ $0,013 \le 0,1$

Найдем вторую пару корней

Выберем $x_0 = -0,5$ $y_0 = -0,6$

$$\begin{cases} (\cos(-1,1) - 1,5)\Delta x + \cos(1,1)\Delta y = \\ = 1,5\cdot(-0,5) - \sin(-1,1) - 0,1 \\ 2\cdot(-0,5)\Delta x + 4(-0,6)\Delta y = 1 - (-0,5)^2 - 2\cdot(-0,6)^2 \end{cases}$$

$\Delta x = -0,04$ $\Delta y = 0,003$

$x_1 = -0,5 - 0,04 = -0,54$

$y_1 = -0,6 + 0,003 \approx -0,6 - 0,59$

$|-0,5 + 0,54| > 0,01$ $|-0,6 + 0,59| \leq 0,01$

Найдем x_2 и y_2

$$\begin{cases} (\cos(-1,13) - 1,5)\Delta x + \cos(1,13)\Delta y = \\ = 0,91 - \sin(-1,13) - 0,1 \\ -1,08\Delta x - 2,36\Delta y = 1 - 0,2916 - 2\cdot 0,3481 \end{cases}$$

$\Delta x = 0,003$ $\Delta y = -0,006$

$x_2 = -0,54 + 0,003 = -0,537$

$y_2 = -0,59 - 0,006 \approx -0,596$

$|-0,54 + 0,537| \leq 0,01$ $|-0,59 + 0,596| \leq 0,01$

Программная реализация задачи

Листинг программы

```
import os

import matplotlib.pyplot as plt
import numpy as np


# NEq
eps = 10 ** -2
funct1 = [(-1.8, 3), (-2.94, 2), (10.37, 1), (5.38, 0)]
funct2 = [(1, 3), (-3.125, 2), (-3.5, 1), (2.458, 0)]
funct3 = [(4.45, 3), (7.81, 2), (-9.62, 1), (-8.17, 0)]
equationNum = "1"
inputFile = "in"
outputFile = "out"
inputDict = {}
inFromFile = False
outToFile = False
leftBound = None
rightBound = None
x_0 = None

# SNEq
u1 = lambda x, y: np.sin(x + y) - 1.5 * x + 0.1
u2 = lambda x, y: x ** 2 + 2 * y ** 2 - 1

v1 = lambda x, y: np.tan(x * y) - x ** 2
v2 = lambda x, y: 0.8 * x ** 2 + 2 * y ** 2 - 1

w1 = lambda x, y: np.tan(x * y + 0.2) - x ** 2
w2 = lambda x, y: x ** 2 + 2 * y ** 2 - 1

systemDict = {
    "1.1": u1,
    "1.2": u2,
    "2.1": v1,
    "2.2": v2,
    "3.1": w1,
    "3.2": w2,
}

systemNum = "1"


def getResNEq(x, n):
    s = "\n\n"
    s += "=" * 100 + "\n"
    s += "Результат работы программы\n"
    s += "=" * 100 + "\n"
    s += f"Корень уравнения x_0 = {x:.3f}\nf(x_0) = {f(x):.3f}\nКоличество
итераций: {n}\n"
    s += "=" * 100 + "\n"
    s += "\n\n"
    return s


def getResSNEq(x, y, n, delta):
    s = "\n\n"
    s += "=" * 100 + "\n"
    s += "Результат работы программы\n"
    s += "=" * 100 + "\n"
    s += f"Вектор неизвестных (x_0, y_0) = ({x:.3f}, {y:.3f})\nКоличество
```

```
итераций: {n}\n"
    s += f"Вектор погрешностей |x_{n}^({n}) - x_{n}^({n - 1})| = {delta:.4f}\n"
    s += "=" * 100 + "\n"
    s += "\n\n"
    return s


# l - коэффициент умножения
# add - слагаемое для прибавления к функции
def f(x, l=1, add=0):
    global equationNum
    res = 0
    # x = round(x, 2)
    if equationNum == "1":
        for tup in funct1:
            # print(f"===\nl={l}, c0={c[0]}, x={x}, c1={c[1]}\n")
            res += l * tup[0] * x ** tup[1]
            # print(f"r = {l * c[0] * x ** c[1]}\n===")
    elif equationNum == "2":
        for tup in funct2:
            res += l * tup[0] * x ** tup[1]
    elif equationNum == "3":
        for tup in funct3:
            res += l * tup[0] * x ** tup[1]
    return res + add


def calcDiff(func, x):
    return (func(x + eps) - func(x)) / eps


def fi(x, l):
    return f(x, l, x)


def simpleIteration(a, b):
    global x_0
    x_curr = a
    l = -1 / max(calcDiff(f, a), calcDiff(f, b))
    n = 0
    while True:
        n += 1
        x_next = fi(x_curr, l)
        if abs(x_next - x_curr) <= eps:
            break
        if (abs(x_next - x_curr)) > 10 or n > 20:
            print("Введите более точные границы, с текущими последовательность
решений не сходится к истинному")
            break
        x_curr = x_next
    x_0 = x_curr
    s = getResNEq(x_curr, n)
    if outToFile:
        with open(outputFile, "w") as file:
            file.write(s)
            print("=" * 100)
            print(f"Решение было записано в файл '{outputFile}'")
            print("=" * 100)
            print("\n")
    else:
        print(s)


def newton(a, b):
```

```
    global x_0
    x_mid = (a + b) / 2
    x_prev = x_mid
    n = 0
    if calcDiff(f, x_mid) > 0:
        x_prev = a
    elif calcDiff(f, x_mid) < 0:
        x_prev = b
    while True:
        n += 1
        h_prev = f(x_prev) / calcDiff(f, x_prev)
        x_curr = x_prev - h_prev
        if abs(x_curr - x_prev) <= eps:
            break
        x_prev = x_curr
    x_0 = x_curr
    s = getResNEq(x_curr, n)
    if outToFile:
        with open(outputFile, "w") as file:
            file.write(s)
            print("=" * 100)
            print(f"Решение было записано в файл '{outputFile}'")
            print("=" * 100)
            print("\n")
    else:
        print(s)


def chords(a, b):
    global x_0
    n = 0
    while True:
        n += 1
        x_curr = a - f(a) * (a - b) / (f(a) - f(b))
        if abs(f(x_curr)) <= eps:
            break
        if f(a) * f(x_curr) < 0:
            b = x_curr
        else:
            a = x_curr
    x_0 = x_curr
    s = getResNEq(x_curr, n)
    if outToFile:
        # open(output_file, 'w').close()
        with open(outputFile, "w") as file:
            file.write(s)
            print("=" * 100)
            print(f"Решение было записано в файл '{outputFile}'")
            print("=" * 100)
            print("\n")
    else:
        print(s)


def solveEquation(methodNum):
    global eps, leftBound, rightBound
    if inFromFile:
        a = float(inputDict["a"])
        b = float(inputDict["b"])
        leftBound = a
        rightBound = b
        if f(a) * f(b) > 0:
            print("На заданном в файле интервале (a, b) нет решений")
            return
```

```
    else:
        while True:
            try:
                print("Введите начальное приближение - (a, b):")
                a = float(input("a = ").strip())
                b = float(input("b = ").strip())
                leftBound = a
                rightBound = b
                if f(a) * f(b) > 0:
                    print("На заданном в файле интервале (a, b) нет решений")
                    continue
                else:
                    break
            except ValueError:
                print("Введите корректное значение!")
        while True:
            try:
                print("Введите погрешность вычислений (epsilon):")
                eps = float(input("epsilon = ").strip())
                break
            except ValueError:
                print("Введите корректное значение!")
    match str(methodNum):
        case "1":
            chords(a, b)
        case "2":
            newton(a, b)
        case "3":
            x = (a + b) / 2
            l = -1 / max(calcDiff(f, a), calcDiff(f, b))
            q = abs(calcDiff(lambda x: fi(x, l), x))
            if not q < 1:
                print(
                    f"На выбранном интервале (a, b) = ({a}, {b})
последовательность решений уравнения не сходится к истинному;")
                while True:
                    try:
                        print("Введите начальное приближение - (a, b):")
                        a = float(input("a = ").strip())
                        b = float(input("b = ").strip())
                        leftBound = a
                        rightBound = b
                        if f(a) * f(b) > 0:
                            print("На заданном в файле интервале (a, b) нет
решений")
                            continue
                        else:
                            simpleIteration(a, b)
                            break
                    except ValueError:
                        print("Введите корректное значение!")
            else:
                simpleIteration(a, b)


def calcDiffX(func, x, y):
    return (func(x + eps, y) - func(x, y)) / eps


def calcDiffY(func, x, y):
    return (func(x, y + eps) - func(x, y)) / eps


def calcDet(A):
```

```
    return A[0][0] * A[1][1] - A[1][0] * A[0][1]


def calcJacobian(func1, func2, x0, y0):
    j11 = calcDiffX(func1, x0, y0)
    j12 = calcDiffY(func1, x0, y0)
    j21 = calcDiffX(func2, x0, y0)
    j22 = calcDiffY(func2, x0, y0)
    return [[j11, j12], [j21, j22]]


def invertMatrix(M):
    c = 1 / calcDet(M)
    return [[c * M[1][1], -1 * c * M[0][1]], [-1 * c * M[1][0], c * M[0][0]]]


def checkSystemSolution(f1, f2, x, y):
    global eps
    return abs(f1(x, y)) <= eps and abs(f2(x, y)) <= eps


def solveSystemOfNonLinearEquationNewton(xVect, yVect):
    x1, x2 = xVect
    y1, y2 = yVect
    x0, y0 = 0, 0
    n = 0
    while True:
        n += 1
        f1 = systemDict[systemNum + ".1"]
        f2 = systemDict[systemNum + ".2"]
        if abs(x2 - x0) <= eps and abs(y2 - y0) <= eps:
            res = getResSNEq(x2, y2, n, abs(x2 - x0))
            if not outToFile:
                print(res)
                if not checkSystemSolution(f1, f2, x2, y2):
                    print("Решение неверное!")
                    print("=" * 100)
            else:
                with open(outputFile, "w") as file:
                    file.write(res)
                    print("=" * 100)
                    print(f"Решение было записано в файл '{outputFile}'")
                    print("=" * 100)
                    print("\n")
            break
        j = calcJacobian(f1, f2, x2, y2)
        j_inv = invertMatrix(j)
        dx = -1 * (f1(x2, y2) * j_inv[0][0] + f2(x2, y2) * j_inv[0][1])
        dy = -1 * (f1(x2, y2) * j_inv[1][0] + f2(x2, y2) * j_inv[1][1])
        x0 = x2
        y0 = y2
        x2 = x2 + dx
        y2 = y2 + dy


def drawGraphSNEq(systemNumber):
    xValues = np.linspace(-1.5, 1.5, 1000)
    yValues = np.linspace(-1.5, 1.5, 1000)
    func1 = systemDict[systemNumber + ".1"]
    func2 = systemDict[systemNumber + ".2"]
    x, y = np.meshgrid(xValues, yValues)
    axes = plt.gca()
    axes.set_xticks(np.arange(-1.5, 1.5, 0.1), minor=True)
    axes.set_yticks(np.arange(-1.5, 1.5, 0.1), minor=True)
```

```
    axes.set_xticks(np.arange(-1.5, 1.5, 0.5))
    axes.set_yticks(np.arange(-1.5, 1.5, 0.5))
    axes.spines['bottom'].set_position('zero')
    axes.spines['left'].set_position('zero')
    axes.text(1.6, 0, 'x')
    axes.text(0, 1.6, 'y')
    zValues1 = func1(x, y)
    zValues2 = func2(x, y)

    plt.contour(x, y, zValues1, levels=[0], colors="red")
    plt.contour(x, y, zValues2, levels=[0], colors="blue")

    plt.axvline(x=0, color='black', linewidth=1)
    plt.axhline(y=0, color='black', linewidth=1)
    plt.grid(True, which="both")
    plt.tight_layout()
    plt.show()


def inputSNEq():
    global systemNum
    if not inFromFile:
        while True:
            print("Выберите систему (1/2/3):")

            print("1: / sin(x + y) = 1,5x - 0,1")
            print("  <")
            print("   \\ x^2 + 2y^2 = 1")
            print("-" * 100)
            print("2: / tg(xy) = x^2")
            print("  <")
            print("   \\ 0,8x^2 + 2y^2 = 1")
            print("-" * 100)
            print("3: / tg(xy + 0,2) = x^2")
            print("  <")
            print("   \\ x^2 + 2y^2 = 1")
            n = input(">>> ").strip()
            if n not in ["1", "2", "3"]:
                print("Введите корректное значение!\n\n")
                continue
            else:
                systemNum = n
                break
        drawGraphSNEq(systemNum)
        x1, x2 = 0, 0
        y1, y2 = 0, 0
        while True:
            try:
                print("Введите начальное приближение по оси x \nx1 <= x0 <= x2")
                x1 = float(input("x1 = ").strip())
                x2 = float(input("x2 = ").strip())
                print("Введите начальное приближение по оси y \ny1 <= y0 <= y2")
                y1 = float(input("y1 = ").strip())
                y2 = float(input("y2 = ").strip())
                break
            except ValueError:
                print("Введите корректное значение!\n\n")
                continue
    else:
        x1, x2 = int(inputDict["x1"]), int(inputDict["x2"])
        y1, y2 = int(inputDict["y1"]), int(inputDict["y2"])
    x = (x1, x2)
    y = (y1, y2)
    solveSystemOfNonLinearEquationNewton(x, y)
```

```
def inputNEq():
    global equationNum, x_0
    if not inFromFile:
        while True:
            print(
                "Выберите уравнение (1/2/3):\n1: -1,8x^3 - 2,94x^2 + 10,37x +
5,38\n2: x^3 - 3,125x^2 - 3,5x + 2,458\n3: 4,45x^3 + 7,81x^2 - 9,62x - 8,17")
            n = input(">>> ").strip()
            if n not in ["1", "2", "3"]:
                print("Введите корректное значение!\n\n")
                continue
            else:
                equationNum = n
                break

        while True:
            print("Каким методом вы хотите решить:\n1 - Метод хорд\n2 - Метод
Ньютона\n3 - Метод простой итерации")
            n = input(">>> ").strip()
            if n not in ["1", "2", "3"]:
                print("Введите корректное значение!\n\n")
                continue
            solveEquation(n)
            break

    else:
        m = inputDict["method"]
        if m not in ["1", "2", "3"]:
            print("Введите корректное значение method в файле!\n\n")
        else:
            solveEquation(m)
    drawGraphNEq()


def userInput():
    global inputFile, outputFile, equationNum, inputDict, inFromFile, outToFile,
systemNum
    while True:
        while True:
            print(
                "Выберите режим работы (1/2/3/4):\n1 - ввод и вывод в консоли\n2
- ввод из файла, вывод в файл\n3 - ввод в консоли, вывод в файл\n4 - ввод из
файла, вывод в консоль")
            n = input(">>> ").strip()
            match n:
                case "1":
                    inFromFile = False
                    outToFile = False
                case "2":
                    inFromFile = True
                    outToFile = True
                case "3":
                    inFromFile = False
                    outToFile = True
                case "4":
                    inFromFile = True
                    outToFile = False
                case _:
                    print("Введите корректное значение!\n\n")
                    continue
            break
        if inFromFile and outToFile:
```

```
            while True:
                inputFile = input("Введите имя файла с входными данными:
").strip()
                outputFile = input("Введите имя файла для записи результата
работы программы: ").strip()
                if not os.path.exists(inputFile):
                    print(f"Не существует такого файла: {inputFile}")
                else:
                    break
        elif inFromFile and not outToFile:
            while True:
                inputFile = input("Введите имя файла с входными данными:
").strip()
                if not os.path.exists(inputFile):
                    print(f"Не существует такого файла: {inputFile}")
                else:
                    break
        while True:
            if not inFromFile:
                while True:
                    print(
                        "Выберите что вы хотите решить (1/2):\n1 - нелинейное
уравнение\n2 - систему нелинейных уравнений")
                    typeOfProgram = input(">>> ").strip()
                    match typeOfProgram:
                        case "1":
                            inputNEq()
                        case "2":
                            inputSNEq()
                        case _:
                            print("Введите корректное значение!\n\n")
            else:
                with open(inputFile) as file:
                    for line in file:
                        key, val = line.split(" = ")[0].strip(), line.split(" =
")[1].strip()
                        inputDict[key] = val
                typeOfProgram = inputDict["typeOfProgram"]
                match typeOfProgram:
                    case "1":
                        equationNum = inputDict["equationNum"]
                        inputNEq()
                        break
                    case "2":
                        systemNum = inputDict["systemNum"]
                        inputSNEq()
                        break
                    case _:
                        print("Введите корректное значение!\n\n")


def drawGraphNEq():
    xValues = np.linspace(leftBound, rightBound, 1000)
    yValues = []
    func = funct1 if equationNum == "1" else (funct2 if equationNum == "2" else
funct3)
    for x in xValues:
        res = 0
        for tup in func:
            res += tup[0] * x ** tup[1]
        yValues.append(res)
    plt.plot(xValues, yValues, color="red", linewidth=1)
    plt.plot(x_0, 0, markersize=4, color="black", marker='o')
    plt.axvline(x=0, color='black', linewidth=1)
```

```
    plt.axhline(y=0, color='black', linewidth=1)
    plt.grid(True)
    plt.tight_layout()
    plt.show()


if __name__ == "__main__":
    userInput()
```

Пример работы программы

```
/usr/local/bin/python3.12 /Users/ipka23/ITMO/2 курс/Вычислительная
математика/lab2/main.py
Выберите режим работы (1/2/3/4):
1 - ввод и вывод в консоли
2 - ввод из файла, вывод в файл
3 - ввод в консоли, вывод в файл
4 - ввод из файла, вывод в консоль
>>> 2
Введите имя файла с входными данными: in
Введите имя файла для записи результата работы программы: out
================================================================================
====================
Решение было записано в файл 'out'
================================================================================
====================


Выберите режим работы (1/2/3/4):
1 - ввод и вывод в консоли
2 - ввод из файла, вывод в файл
3 - ввод в консоли, вывод в файл
4 - ввод из файла, вывод в консоль
>>> 1
Выберите что вы хотите решить (1/2):
1 - нелинейное уравнение
2 - систему нелинейных уравнений
>>> 1
Выберите уравнение (1/2/3):
1: -1,8x^3 - 2,94x^2 + 10,37x + 5,38
2: x^3 - 3,125x^2 - 3,5x + 2,458
3: 4,45x^3 + 7,81x^2 - 9,62x - 8,17
>>> 1
Каким методом вы хотите решить:
1 - Метод хорд
2 - Метод Ньютона
3 - Метод простой итерации
>>> 1
Введите начальное приближение - (a, b):
a = -1
b = 0
Введите погрешность вычислений (epsilon):
epsilon = 0.01


================================================================================
====================
Результат работы программы
================================================================================
====================
Корень уравнения x_0 = -0.474
f(x_0) = -0.002
Количество итераций: 2
================================================================================
====================
```

```
Выберите что вы хотите решить (1/2):
1 - нелинейное уравнение
2 - систему нелинейных уравнений
>>> 2
Выберите систему (1/2/3):
1: / sin(x + y) = 1,5x - 0,1
  <
   \ x^2 + 2y^2 = 1
--------------------------------------------------------------------------------
--------------------
2: / tg(xy) = x^2
  <
   \ 0,8x^2 + 2y^2 = 1
--------------------------------------------------------------------------------
--------------------
3: / tg(xy + 0,2) = x^2
  <
   \ x^2 + 2y^2 = 1
>>> 1
Введите начальное приближение по оси x
x1 <= x0 <= x2
x1 = 0
x2 = 1
Введите начальное приближение по оси y
y1 <= y0 <= y2
y1 = 0
y2 = 1


================================================================================
====================
Результат работы программы
================================================================================
====================
Вектор неизвестных (x_0, y_0) = (0.688, 0.513)
Количество итераций: 5
Вектор погрешностей |x_5^(5) - x_5^(4)| = 0.0003
================================================================================
====================
```

Вывод

Данная лабораторная помогла понять, как решаются нелинейные уравнения и системы нелинейных уравнений численными методами